

# Front End con Angular



## FASE 1 — LISTAR EMPLEADOS (Solo lectura)

---

### 1. Crear proyecto

En tu terminal:

```
cd C:\Cursos\Angular  
ng new rh-app
```

Responde:

- Would you like to add Angular routing? → Yes
- Which stylesheet format would you like to use? → CSS

Luego:

```
cd rh-app
```

---

## 2. Agregar Bootstrap (modo oscuro)

### Archivo a modificar

src/index.html

Reemplaza **TODO** su contenido por:

```
<!doctype html>
<html lang="en" data-bs-theme="dark">
<head>
  <meta charset="utf-8">
  <title>RH App</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min
.css" rel="stylesheet">

</head>
<body>
  <app-root></app-root>

  <!-- Bootstrap JS -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundl
e.min.js"></script>
</body>
</html>
```

**Por qué:** Activamos Bootstrap por CDN y forzamos **modo oscuro global**.

---

## 3. Modelo de datos

### Crear archivo

src/app/modelos/empleado.ts

```
export interface Empleado {
  idEmpleado: number;
  nombre: string;
  departamento: string;
  sueldo: number;
}
```

**Por qué:** Representa la estructura exacta del backend.



## 4. Servicio EmpleadoServicio (con inject)

```
ng g s servicios/empleado --skip-tests
```



### Archivo

```
src/app/servicios/empleado.service.ts
```

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Empleado } from '../../modelos/empleado';

@Injectable({ providedIn: 'root' })
export class EmpleadoServicio {

  private http = inject(HttpClient);
  private baseUrl = 'http://localhost:8080/api/empleados';

  obtenerEmpleados(): Observable<Empleado[]> {
    return this.http.get<Empleado[]>(this.baseUrl);
  }

  obtenerEmpleadoPorId(id: number): Observable<Empleado> {
    return this.http.get<Empleado>(`${this.baseUrl}/${id}`);
  }

  agregarEmpleado(empleado: Empleado): Observable<Empleado> {
    const { idEmpleado, ...sinId } = empleado;
    return this.http.post<Empleado>(this.baseUrl, sinId);
  }

  editarEmpleado(id: number, empleado: Empleado): Observable<Empleado> {
    return this.http.put<Empleado>(`${this.baseUrl}/${id}`, empleado);
  }

  eliminarEmpleado(id: number): Observable<{ message: string }> {
    return this.http.delete<{ message: string }>(`${this.baseUrl}/${id}`);
  }
}
```

**Por qué:** Servicio centralizado con `inject()` y tipado completo.



## 5. Componente Lista

```
ng g c empleados/lista --skip-tests
```

**src/app/empleados/lista/lista.component.ts**

```
import { Component, inject, signal } from '@angular/core';
import { EmpleadoServicio } from '../../../servicios/empleado.service';
import { Empleado } from '../../../modelos/empleado';

@Component({
  selector: 'app-lista',
  standalone: true,
  templateUrl: './lista.component.html'
})
export class ListaComponent {

  empleados = signal<Empleado[]>([]);
  private empleadoService = inject(EmpleadoServicio);

  ngOnInit() {
    this.cargar();
  }

  cargar() {
    this.empleadoService.obtenerEmpleados()
      .subscribe(data => this.empleados.set(data));
  }
}
```

**src/app/empleados/lista/lista.component.html**

```
<div class="container mt-4">
  <h2>Listado de Empleados</h2>

  <table class="table table-dark table-striped mt-3">
    <thead>
      <tr>
        <th>ID</th>
        <th>Nombre</th>
        <th>Departamento</th>
        <th>Sueldo</th>
      </tr>
    </thead>
    <tbody>
      @for (emp of empleados(); track emp.idEmpleado) {
        <tr>
          <td>{{ emp.idEmpleado }}</td>
          <td>{{ emp.nombre }}</td>
          <td>{{ emp.departamento }}</td>
          <td>{{ emp.sueldo }}</td>
        </tr>
      }
    </tbody>
  </table>
</div>
```

**Por qué:** Uso de **signals** + **@for** (Angular moderno).

## 6. Rutas

 **src/app/app.routes.ts**

```
import { Routes } from '@angular/router';
import { ListaComponent } from '../empleados/lista/lista.component';

export const routes: Routes = [
  { path: 'empleados', component: ListaComponent },
  { path: '', redirectTo: 'empleados', pathMatch: 'full' }
];
```

## 7. Configuración principal

 **src/app/app.config.ts**

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

## 8. Componente raíz

 **src/app/app.component.ts**

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html'
})
export class AppComponent {}
```

 **src/app/app.component.html**

```
<nav class="navbar navbar-expand-lg bg-dark border-bottom">
  <div class="container">
    <a class="navbar-brand">RH App</a>
    <div class="navbar-nav">
      <a class="nav-link" routerLink="/empleados">Listado</a>
      <a class="nav-link disabled">Agregar</a>
    </div>
  </div>
</nav>

<router-outlet></router-outlet>
```

---



## Prueba rápida

ng serve

Visita:

👉 <http://localhost:4200/empleados>

Deberías ver la tabla cargada desde el backend.

---



## Solución de problemas

### CORS:

Si no carga, en el backend agrega:

```
@CrossOrigin(origins = "http://localhost:4200")
```

### Error de HttpClient:

Verifica que `provideHttpClient()` esté en `app.config.ts`.

---

Cuando todo esto esté funcionando, respóndeme:

listo

y continuamos con **FASE 2 — Agregar Empleado + Formularios + Navbar completo**



listo vamos a la fase 2

Perfecto 🙌

Vas avanzando exactamente como se hace en proyectos profesionales.



## FASE 2 — AGREGAR EMPLEADO

*(Formulario + ruta + navbar completo)*



### 1. Generar componente Agregar

En terminal:

```
ng g c empleados/agregar --skip-tests
```



### 2. Configurar rutas

Archivo: `src/app/app.routes.ts`

Reemplaza por:

```
import { Routes } from '@angular/router';
import { ListaComponent } from '../empleados/lista/lista.component';
import { AgregarComponent } from '../empleados/agregar/agregar.component';

export const routes: Routes = [
  { path: 'empleados', component: ListaComponent },
  { path: 'agregar-empleado', component: AgregarComponent },
  { path: '', redirectTo: 'empleados', pathMatch: 'full' }
];
```

Por qué: Registramos la nueva vista del formulario.



### 3. Actualizar Navbar

Archivo: `src/app/app.component.html`

Reemplaza por:

```
<nav class="navbar navbar-expand-lg bg-dark border-bottom">
  <div class="container">
    <a class="navbar-brand">RH App</a>
    <div class="navbar-nav">
```

```
        <a class="nav-link" routerLink="/empleados">Listado</a>
        <a class="nav-link" routerLink="/agregar-empleado">Agregar</a>
      </div>
    </div>
  </nav>

  <router-outlet></router-outlet>
```

**Por qué:** Ahora puedes navegar entre vistas.

---

## 4. Lógica del componente Agregar

**Archivo:** `src/app/empleados/agregar/agregar.component.ts`

```
import { Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { EmpleadoServicio } from '../../../servicios/empleado.service';
import { Empleado } from '../../../modelos/empleado';

@Component({
  selector: 'app-agregar',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './agregar.component.html'
})
export class AgregarComponent {

  private empleadoService = inject(EmpleadoServicio);
  private router = inject(Router);

  empleado: Empleado = {
    idEmpleado: 0,
    nombre: '',
    departamento: '',
    sueldo: 0
  };

  guardar() {
    this.empleadoService.agregarEmpleado(this.empleado)
      .subscribe(() => this.router.navigate(['/empleados']));
  }
}
```

**Por qué:**

Formulario simple con **template-driven forms** y navegación al guardar.

---





## 5. Vista del formulario

**Archivo:** `src/app/empleados/agregar/agregar.component.html`

```

<div class="container mt-4">
  <h2>Agregar Empleado</h2>

  <form (ngSubmit)="guardar()">

    <div class="mb-3">
      <label class="form-label">Nombre</label>
      <input class="form-control" [(ngModel)]="empleado.nombre"
name="nombre">
    </div>

    <div class="mb-3">
      <label class="form-label">Departamento</label>
      <input class="form-control" [(ngModel)]="empleado.departamento"
name="departamento">
    </div>

    <div class="mb-3">
      <label class="form-label">Sueldo</label>
      <input type="number" class="form-control"
[(ngModel)]="empleado.sueldo" name="sueldo">
    </div>

    <button class="btn btn-success">Guardar</button>
  </form>
</div>

```

---



## Prueba rápida

1. Ve a: `http://localhost:4200/agregar-empleado`
  2. Registra un empleado
  3. Automáticamente vuelve al listado
  4. Confirma que aparece en la tabla
- 



## Problemas comunes

### Formulario no enlaza datos

→ Verifica que `FormsModule` esté en `imports`.

### POST no funciona

→ Confirma endpoint:

POST `http://localhost:8080/api/empleados`

---

Cuando todo esté funcionando responde:

listo

y entramos a **FASE 3 — Editar empleado** 🧑✍️

listo vamos a la fase 3

Excelente 🍀

Tu aplicación ya se ve como un sistema real de RH.

Vamos con la parte más interesante:

---

## **FASE 3 — EDITAR EMPLEADO** (carga por :id)

---

### **1. Generar componente Editar**

```
ng g c empleados/editar --skip-tests
```

---

### **2. Agregar ruta de edición**

**Archivo:** `src/app/app.routes.ts`

Reemplázalo por:

```
import { Routes } from '@angular/router';
import { ListaComponent } from '../empleados/lista/lista.component';
import { AgregarComponent } from '../empleados/agregar/agregar.component';
import { EditarComponent } from '../empleados/editar/editar.component';

export const routes: Routes = [
  { path: 'empleados', component: ListaComponent },
  { path: 'agregar-empleado', component: AgregarComponent },
  { path: 'editar-empleado/:id', component: EditarComponent },
  { path: '', redirectTo: 'empleados', pathMatch: 'full' }
];
```

**Por qué:**

Ahora Angular puede recibir el `id` desde la URL.

### 3. Botón Editar en la lista

**Archivo:** `src/app/empleados/lista/lista.component.html`

Reemplaza el `<thead>` y `<tbody>` por:

```
<thead>
  <tr>
    <th>ID</th>
    <th>Nombre</th>
    <th>Departamento</th>
    <th>Sueldo</th>
    <th>Acciones</th>
  </tr>
</thead>
<tbody>
  @for (emp of empleados(); track emp.idEmpleado) {
    <tr>
      <td>{{ emp.idEmpleado }}</td>
      <td>{{ emp.nombre }}</td>
      <td>{{ emp.departamento }}</td>
      <td>{{ emp.sueldo }}</td>
      <td>
        <a class="btn btn-warning btn-sm"
          [routerLink]="['/editar-empleado', emp.idEmpleado]">
          Editar
        </a>
      </td>
    </tr>
  }
</tbody>
```

### 4. Lógica del componente Editar

**Archivo:** `src/app/empleados/editar/editar.component.ts`

```
import { Component, inject } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { FormsModule } from '@angular/forms';
import { EmpleadoServicio } from '../../../servicios/empleado.service';
import { Empleado } from '../../../modelos/empleado';

@Component({
  selector: 'app-editar',
  standalone: true,
```

```

    imports: [FormsModule],
    templateUrl: './editar.component.html'
  })
  export class EditarComponent {

    private empleadoService = inject(EmpleadoServicio);
    private route = inject(ActivatedRoute);
    private router = inject(Router);

    empleado!: Empleado;

    ngOnInit() {
      const id = Number(this.route.snapshot.paramMap.get('id'));
      this.empleadoService.obtenerEmpleadoPorId(id)
        .subscribe(data => this.empleado = data);
    }

    guardar() {
      this.empleadoService.editarEmpleado(this.empleado.idEmpleado,
      this.empleado)
        .subscribe(() => this.router.navigate(['/empleados']));
    }
  }

```

**Por qué:**

Se carga el empleado desde la URL y se reutiliza el mismo formulario para editar.



## 5. Vista del formulario de edición

**Archivo:** `src/app/empleados/editar/editar.component.html`

```

<div class="container mt-4" *ngIf="empleado">
  <h2>Editar Empleado</h2>

  <form (ngSubmit)="guardar()">

    <div class="mb-3">
      <label class="form-label">Nombre</label>
      <input class="form-control" [(ngModel)]="empleado.nombre"
name="nombre">
    </div>

    <div class="mb-3">
      <label class="form-label">Departamento</label>
      <input class="form-control" [(ngModel)]="empleado.departamento"
name="departamento">
    </div>

    <div class="mb-3">
      <label class="form-label">Sueldo</label>
      <input type="number" class="form-control"
[(ngModel)]="empleado.sueldo" name="sueldo">

```

```
</div>

<button class="btn btn-primary">Actualizar</button>
</form>
</div>
```

---

## Prueba rápida

1. Ve a `http://localhost:4200/empleados`
  2. Pulsa **Editar** en cualquier fila
  3. Cambia datos y guarda
  4. Vuelve al listado con los cambios reflejados
- 

## Problemas comunes

### No carga el formulario

→ Revisa que el backend tenga:

GET /empleados/{id}

### No se actualiza

→ Revisa PUT /empleados/{id}

---

Cuando esté funcionando responde:

listo

y entramos a la **FASE 4 — Eliminar empleado + confirmación + refrescar lista** 🗑️ ✨

listo, vamos a la fase 4

Perfecto 💪

Ya tienes un **CRUD casi completo de nivel profesional**.

Vamos a cerrarlo correctamente.

---

## FASE 4 — ELIMINAR EMPLEADO (confirm + refrescar lista)

## 1. Agregar botón Eliminar en la lista

### Archivo:

src/app/empleados/lista/lista.component.html

Dentro de la columna **Acciones**, deja así:

```
<td>
  <a class="btn btn-warning btn-sm me-2"
    [routerLink]="['/editar-empleado', emp.idEmpleado]">
    Editar
  </a>

  <button class="btn btn-danger btn-sm"
    (click)="eliminar(emp.idEmpleado)">
    Eliminar
  </button>
</td>
```

## 2. Lógica de eliminación

### Archivo:

src/app/empleados/lista/lista.component.ts

Agrega este método dentro de la clase:

```
eliminar(id: number) {
  const ok = confirm('¿Deseas eliminar este empleado?');
  if (!ok) return;

  this.empleadoService.eliminarEmpleado(id)
    .subscribe(() => this.cargar());
}
```

Por qué:

- `confirm()` evita eliminaciones accidentales
- `this.cargar()` refresca la lista automáticamente

## Prueba rápida

1. Ve a `http://localhost:4200/empleados`
2. Pulsa **Eliminar**
3. Confirma
4. Observa cómo se borra y la tabla se actualiza sin recargar la página



## Problemas comunes

### No elimina

→ Verifica endpoint:

DELETE `http://localhost:8080/api/empleados/{id}`

### La tabla no se actualiza

→ Asegúrate de llamar `this.cargar()` después del delete.

---

# ❖ RESULTADO FINAL

Ahora tienes:

- ✓ Listar empleados
- ✓ Agregar empleados
- ✓ Editar empleados
- ✓ Eliminar empleados
- ✓ Angular moderno (signals, inject, @for)
- ✓ Bootstrap oscuro
- ✓ Arquitectura limpia y escalable

Este proyecto ya se puede usar como **base real de RH** y como portafolio 🧰 🚀

Saludos!

**Ing. Ubaldo Acosta**

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)